

Тема 10 – Алгоритми за търсене и за хеширане

1. Алгоритми за търсене (линейно търсене, двоично търсене).
2. Алгоритми за хеширане. Хеш-функции. Обработка на колизии.

Тема 10 – Алгоритми за търсене

Линейното търсене е най-простият алгоритъм за търсене. При него елементът се търси последователно в списъка. Този метод може да бъде изпълнен върху сортиран или несортиран списък. В случай на сортиран списък търсенето започва от 0-ия елемент и продължава, докато елементът е намерен в списъка или се достигне елемента, чиято стойност е по-голяма от търсената (ако приемем, че списъкът е сортиран в възходящ ред). Търсенето в случай на несортиран списък също започва от 0-ия елемент и продължава до достигане на елемента или края на списъка. Алгоритъмът за линейно търсене търси последователно всички елементи в масива.

Най-доброто му време за изпълнение е 1, докато най-лошото време за изпълнение е n , където n е общото брой елементи в масива за търсене. Това е най-простият алгоритъм за търсене в списък от данните, който често се предпочита, когато данните не са сортирани.

Тема 10 – Алгоритми за търсене

Линейното търсене се дефинира като алгоритъм за последователно търсене, който започва от единия край и преминава през всеки елемент от списъка, докато се намери желаният елемент, в противен случай търсенето продължава до края на набора от данни. Това е най-лесният алгоритъм за търсене.

Времевата сложност на линейното търсене е $O(n)$.

Друг подход за търсене в списък е **двоично търсене**. Двоичното търсене е алгоритъм за търсене, използван в **сортиран масив** чрез многократно разделяне на интервала на търсене наполовина. Идеята на двоичното търсене е да се използва информацията, че масивът е сортиран и да се намали времевата сложност до **$O(\log n)$** .

Тема 10 – Алгоритми за търсене

Алгоритъм за двоично търсене: Основните стъпки за извършване на двоично търсене са:

1. Сортирайте масива във възходящ ред.
2. Задайте ниския индекс на първия елемент от масива и високия индекс на последния елемент.
3. Задайте средния индекс като средна стойност на ниския и високия индекси.
4. Ако елементът в средния индекс е целевият елемент, върнете средния индекс.
5. Ако целевият елемент е по-малък от елемента със среден индекс, задайте високия индекс на среден индекс – 1.
6. Ако целевият елемент е по-голям от елемента със средния индекс, задайте ниския индекс на средния индекс + 1.
7. Повторете стъпки, докато елементът бъде намерен или стане ясно, че елементът не присъства в масива.

Времева сложност: **$O(\log n \text{ база } 2)$**

Тема 10 – Алгоритми за търсене

Алгоритъмът за двоично търсене може да бъде реализиран по следните два начина:

1. Итеративен метод
2. Рекурсивен метод

Итеративен метод (псевдокод):

`binarySearch(arr, x, low, high)`

repeat till `low = high`

`mid = (low + high)/2`

 if (`x == arr[mid]`)

 return `mid`

 else if (`x > arr[mid]`) // `x` is on the right side

`low = mid + 1`

 else // `x` is on the left side

`high = mid - 1`

Тема 10 – Алгоритми за търсене

Рекурсивен метод (псевдокод):

```
binarySearch(arr, x, low, high)
    if low > high
        return False
    else
        mid = (low + high) / 2
        if x == arr[mid]
            return mid
        else if x > arr[mid]    // x is on the right side
            return binarySearch(arr, x, mid + 1, high)
        else                  // x is on the left side
            return binarySearch(arr, x, low, mid - 1)
```

Тема 10 – Алгоритми за търсене

Binary Search

Search 23

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

23 > 16
take 2nd half

L=0 1 2 3 M=4 5 6 7 8 H=9

2	5	8	12	16	23	38	56	72	91
---	---	---	----	----	----	----	----	----	----

23 < 56
take 1st half

0 1 2 3 4 L=5 6 M=7 8 H=9

2	5	8	12	16	23	38	56	72	91
---	---	---	----	----	----	----	----	----	----

Found 23,
Return 5

0	1	2	3	4	L=5, M=5	H=6	7	8	9
2	5	8	12	16	23	38	56	72	91

Тема 10 – Алгоритми за търсене

Тройното търсене (Ternary search) е алгоритъм за намаляване (по константа) и завладяване, който може да се използва за намиране на елемент в масив. Подобно е на двоично търсене, където разделяме масива на две части, но в този алгоритъм разделяме дадения масив на три части и определяме коя има ключа (търсения елемент). Можем да разделим масива на три части, като вземем $mid1$ и $mid2$, които трябва да бъдат изчислени. Първоначално $l=0$ и $r= n-1$, където n е дължината на масива.

$$mid1 = l + (r-l)/3$$

$$mid2 = r - (r-l)/3$$

Единствената разлика от двоичното търсене е, че намалява времевата сложност на **$O(\log n \text{ база } 3)$** , а тази на двоичното търсене е **$O(\log n \text{ база } 2)$** .

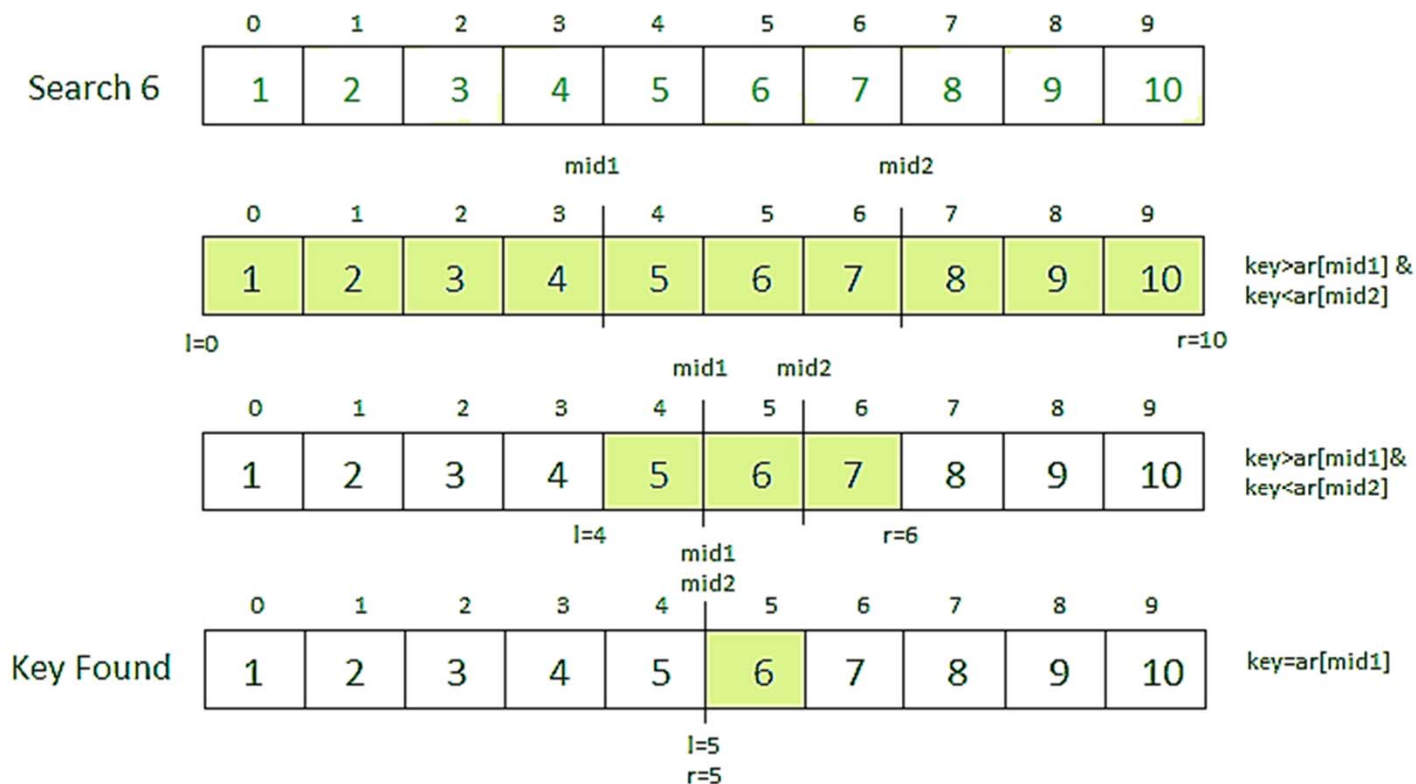
Тема 10 – Алгоритми за търсене

Забележка: Масивът трябва да бъде сортиран, за да се извърши троично търсене върху него.

Стъпки за извършване на троично търсене:

1. Първо сравняваме ключа (търсената стойност) с елемента в $mid1$. Ако се намери равно, връщаме $mid1$.
2. Ако не, тогава сравняваме ключа с елемента в $mid2$. Ако се намери равно, връщаме $mid2$.
3. Ако не, тогава проверяваме дали ключът е по-малък от елемента в $mid1$. Ако да, върнете се към първата част.
4. Ако не, тогава проверяваме дали ключът е по-голям от елемента в $mid2$. Ако да, върнете се към третата част.
5. Ако не, тогава се връщаме към втората (средна) част.

Тема 10 – Алгоритми за търсене



Тема 10 – Алгоритми за хеширане

Алгоритми за хеширане.

Хеш-функции.

Обработка на колизии.

Тема 10 – Алгоритми за хеширане

Алгоритъмът за хеширане е математическа функция, която изкривява данните и ги прави нечетими. Алгоритмите за хеширане са еднопосочни програми, така че текстът не може да бъде разкодиран и декодиран от някой друг.

Алгоритъмът за хеширане е криптографска хеш функция. Това е математически алгоритъм, който преобразува данни с произволен размер в хеш с фиксиран размер.

Тема 10 – Алгоритми за хеширане

Необходимост от хеш структура на данните

Смисълът за използване на хеш структура на данни е в постигане на по-висока ефективност по време при търсене на данни. При съхраняването на данни в масив сложността по време е $O(1)$, но при търсене на данни в масив времевата сложност е $O(\log n)$. За голям набор от данни може да причини много проблеми и това от своя страна прави структурата на данните на масива неефективна.

При хеш структура на данни обаче съхраняването и търсенето на данните е постоянно време, т.е. за има времева сложност $O(1)$, тоест е възможно лесно да се съхраняват данни в постоянно време и да се извличат също в постоянно време.

Тема 10 – Алгоритми за хеширане

Компоненти на хеширането

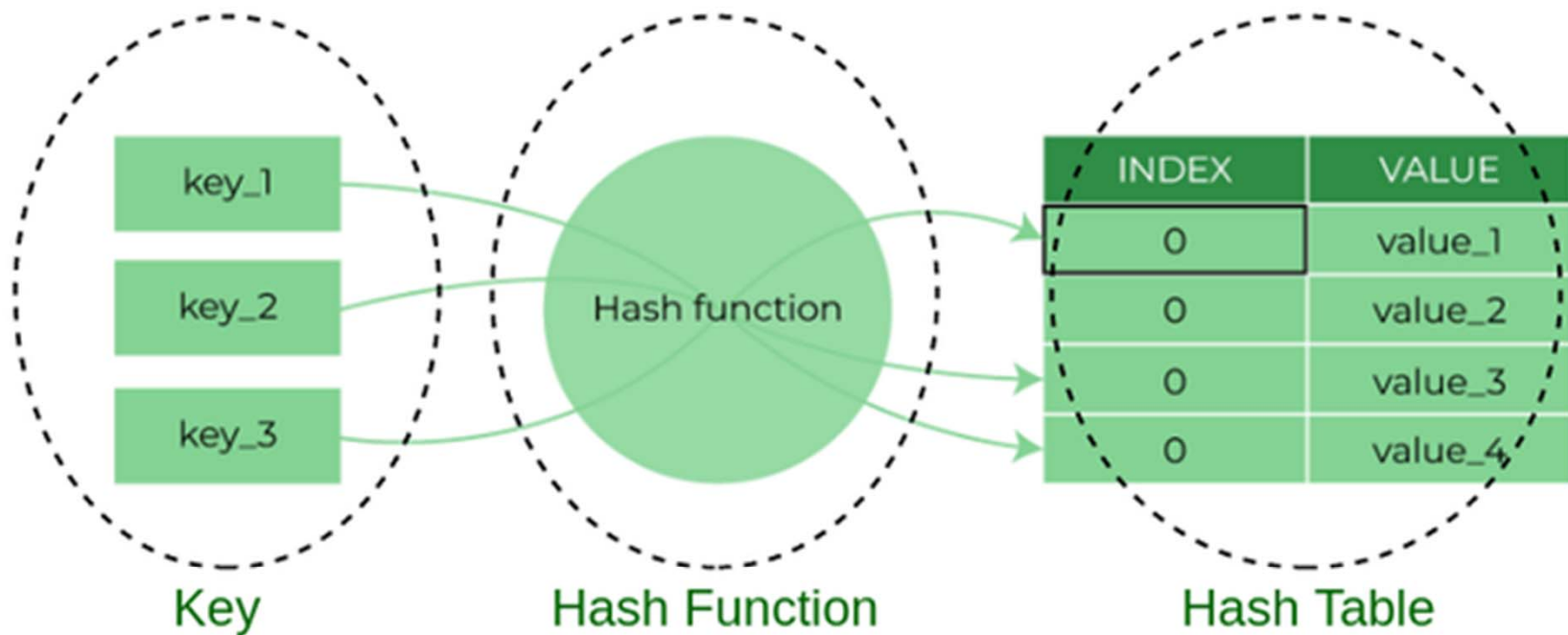
Има основно три компонента на хеширането:

Ключ: Ключът може да бъде всеки низ или цяло число, което се подава като вход в хеш функцията, която определя индекс или местоположение за съхранение на елемента в структура от данни.

Хеш функция: Хеш функцията получава входния ключ и връща индекса на елемент в масив, наречен хеш таблица. Индексът е известен като хеш индекс.

Хеш таблица: Хеш таблицата е структура от данни, която съпоставя ключове към стойности с помощта на специална функция, наречена хеш функция. Хешът съхранява данните по асоциативен начин в масив, където всяка стойност на данните има свой собствен уникален индекс.

Тема 10 – Алгоритми за хеширане



Components of Hashing

Тема 10 – Алгоритми за хеширане

Как работи хеширането?

Да предположим, че имаме набор от низове {"ab", "cd", "efg"} и бихме искали да го съхраним в таблица.

Основната ни цел тук е да търсим или актуализираме стойностите, съхранени в таблицата, бързо за $O(1)$ време и не сме загрижени за подреждането на низовете в таблицата. Така даденият набор от низове може да действа като ключ и самият низ ще действа като стойността на низа, но как да съхраним стойността, съответстваща на ключа?

Стъпка 1: Знаем, че хеш функциите (което е някаква математическа формула) се използват за изчисляване на хеш стойността, която действа като индекс на структурата от данни, където ще се съхранява стойността.

Стъпка 2: И така, нека възложим

"a" = 1,

"b" = 2, .. и т.н., към всички азбучни знаци.

Стъпка 3: Следователно числената стойност чрез сумиране на всички символи на низа:

$$„ab“ = 1 + 2 = 3,$$

$$“cd” = 3 + 4 = 7,$$

$$“efg” = 5 + 6 + 7 = 18$$

Тема 10 – Алгоритми за хеширане

Стъпка 4: Сега приемерте, че имаме таблица с размер 7 за съхраняване на тези низове. Хеш функцията, която се използва тук, е сумата от знаците в ключовия размер на таблицата. Можем да изчислим местоположението на низа в масива, като вземем $\text{sum}(\text{string}) \bmod 7$.

Стъпка 5: След това ще съхраняваме

„ab“ в $3 \bmod 7 = 3$,

“cd” в $7 \bmod 7 = 0$ и

„efg“ в $18 \bmod 7 = 4$.

0	1	2	3	4	5	6
cd			ab	efg		

Ключ за съпоставяне с индекси на масив

Горната техника ни позволява да изчислим местоположението на даден низ с помощта на проста хеш функция и бързо да намерим стойността, която се съхранява на това място. Следователно идеята за хеширане изглежда като добър начин за съхраняване на двойки (ключ, стойност) от данните в таблица.

Тема 10 – Алгоритми за хеширане

Криптографската хеш функция – изисквания към нея:

- * Трябва бързо да се изчисли хеш стойността за всякакъв вид данни;
- * Трябва да е невъзможно повторно генериране на съобщението от неговата хеш стойност;
- * Трябва да е невъзможно да се намерят две съобщения с еднакъв хеш (колизия);
- * Всяка промяна на съобщението, дори и най-малката, трябва да промени хеш стойността.

Тема 10 – Алгоритми за хеширане

Добрите хеш функции трябва да имат следните свойства:

Ефективно изчислими.

1. Трябва да разпреди равномерно ключовете (Всяка позиция в таблицата да е еднакво вероятна за всеки ключ).
2. Трябва да минимизира колизиите (сблъсъците).
3. Трябва да има нисък коефициент на натоварване (броят елементи в таблицата, разделен на размера на таблицата).

Сложност на изчисляването на хеш стойността с помощта на хеш функцията

Времева сложност: $O(n)$

Пространствена сложност: $O(1)$

Тема 10 – Алгоритми за хеширане

Криптографските хеш функции се използват широко в ИТ. Можем да ги използваме за цифрови подписи, кодове за удостоверяване на съобщения и други форми на удостоверяване. Можем също да ги използваме за индексване на данни в хеш-таблицы, за пръстови отпечатъци, идентифициране на файлове, откриване на дубликати или като контролни суми (можем да открием дали изпратен файл не е претърпял случайно или умишлено повреда на данните).

Можем да ги използваме и за съхранение на пароли, без да е необходимо да съхраняваме паролата на потребителите. Просто трябва да се провери дали паролата на потребителя и паролата на всеки даден опит съвпадат, така че хешовете трябва да работят добре и да предоставят допълнителна защита на потребителите.

Тема 10 – Алгоритми за хеширане

MD5 хеш функция

Този алгоритъм за хеширане беше широко използван и все още е един от най-широко известните алгоритми за хеширане. Но въпреки че първоначално е проектиран да се използва като функция на криптографски алгоритъм, той вече не се счита за безопасен за използване за криптографски цели, тъй като е компрометиран.

Когато MD5 се използва за директно хеширане на пароли, има още по-лесен начин да го разбиете... Google. Като напишете хеша в полето за търсене, има голям шанс да получите предишното му състояние в рамките на милисекунди!

Тема 10 – Алгоритми за хеширане

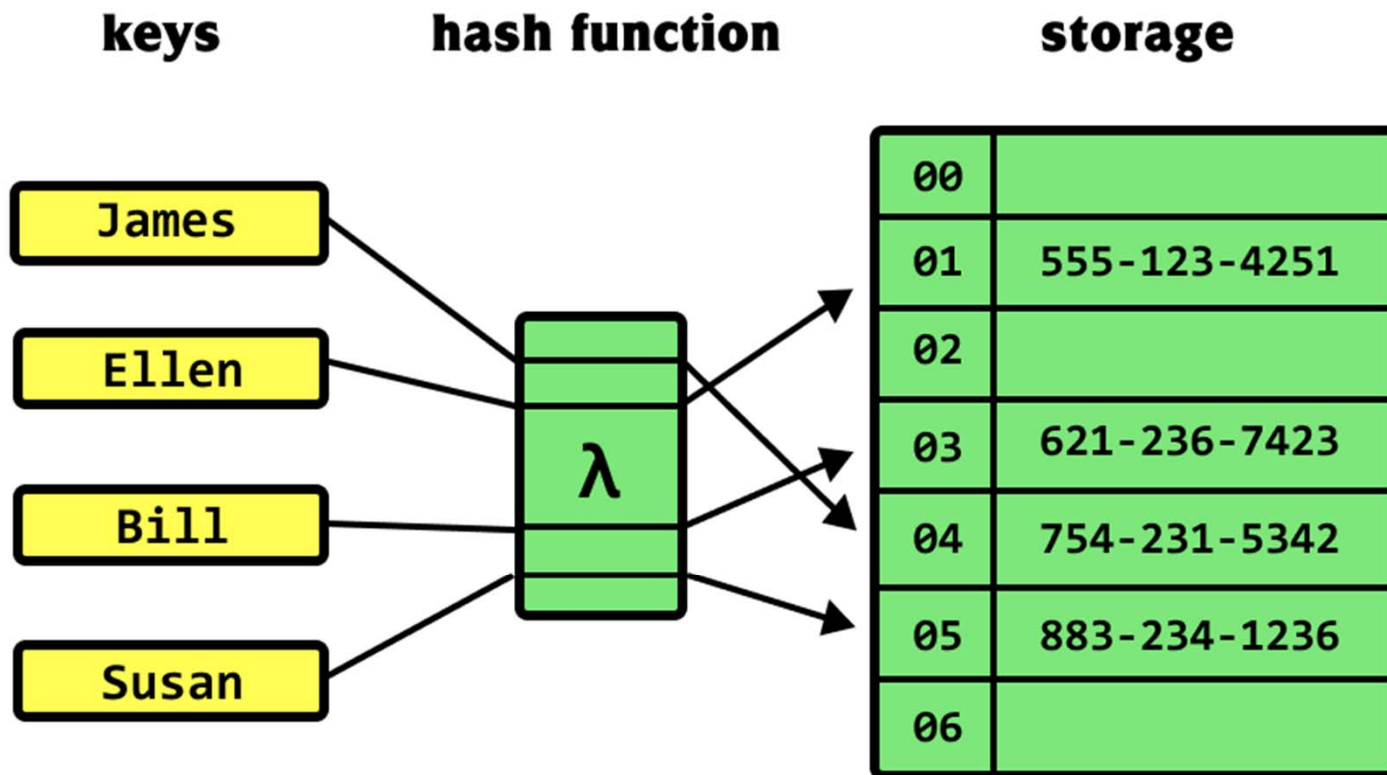
Хеш таблицата е структура от данни, съдържаща **ключ** и **данни**, която се характеризира с директен достъп до елементите, независимо от типа им.

Ключът е уникален за разлика от данните – **не може да съществуват два елемента с един и същи ключ.**

Сложността на елементарните операции по ключ (търсене, вмъкване, изтриване и актуализиране) в общия случай е константна, което я прави изключително полезна.

Хеш таблицата използва хеш функция за изчисляване на индекс, наричан още **хеш код**, в масив от “кофи” или слотове, от които може да бъде намерена желаната стойност. По време на търсенето ключът се хешира и полученият хеш показва къде се съхранява съответната стойност.

Тема 10 – Алгоритми за хеширане



Тема 10 – Алгоритми за хеширане

Приложение

При процеса на компилиране на дадена програма всеки идентификатор в нея се съхранява в определена структура (таблица), в която за него има полета и за друга информация – тип, адрес от паметта и други. Когато се срещне такъв идентификатор трябва да се определи дали той вече не е сред множеството идентификатори, които вече са в таблицата.

Последователното обхождане и поддържането в сортиран вид на тези структури са бавни операции. Най-удачна реализация на операциите търсене, четене и вмъкване в такова множество от идентификатори (ключове) е хеширането (от англ. „hash, hashing“).

Хеширането е много добър пример за компромисно решение на проблема памет-време. При хеширането ключът се преобразува чрез определена хеш функция в адрес на таблица (хеш таблица) т.е. на дадени данни се съпоставят други, посредством правила, но така че обратната операция да е невъзможна (ако хеширате даден низ, с хеш функция, то от резултата няма как да получите обратно първоначалния низ).

Тема 10 – Алгоритми за хеширане

Основните операции за работа с хеш таблица са следните три:

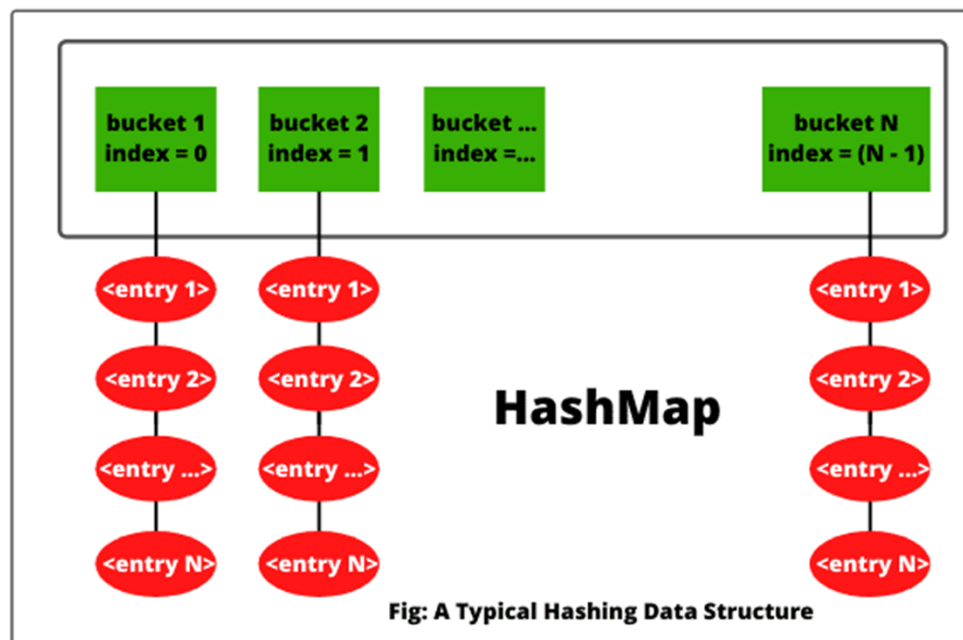
- * **Void put (key, data);** – включване на елемент с ключ (key) и данни (data) в хеш таблицата.
- * **Data get (key);** – търсене и извличане на данните от елемент с ключ (key) от хеш таблицата.
- * **Void remove (key);** – изключване на елемент с ключ (key) от хеш таблицата.

Обобщение:

Всеки елемент на хеш таблицата се характеризира с две полета: ключ (key) и данни (data). Ключът представлява уникален идентификатор: ако два елемента имат един и същ ключ се считат идентични.

Тема 10 – Алгоритми за хеширане

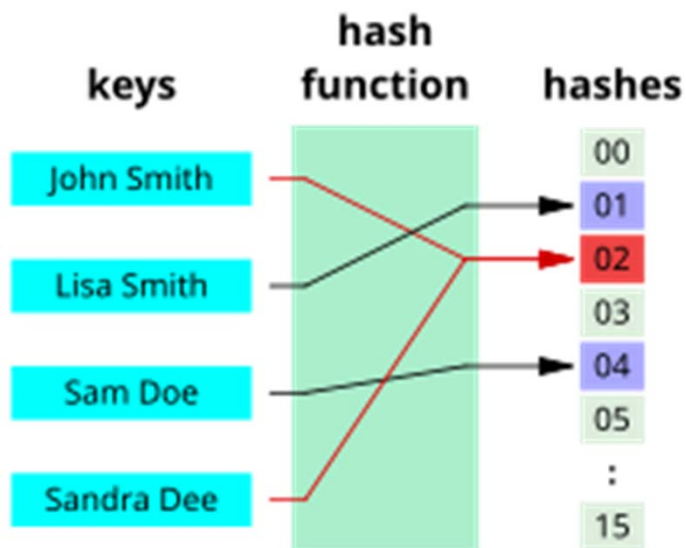
Една от основните разлики между **HashMap** и **Hashtable** е, че **HashMap** не е синхронизиран, докато **Hashtable** е синхронизиран, което означава, че **Hashtable** е безопасен за многонишкова работа. Тоест **Hashtable** може да се споделя между множество нишки, за разлика от **HashMap**, който не може да се споделя между множество нишки без подходяща синхронизация.



Тема 10 – Алгоритми за хеширане

Колизия наричаме ситуация, при която два различни ключа връщат едно и също число за хеш код.

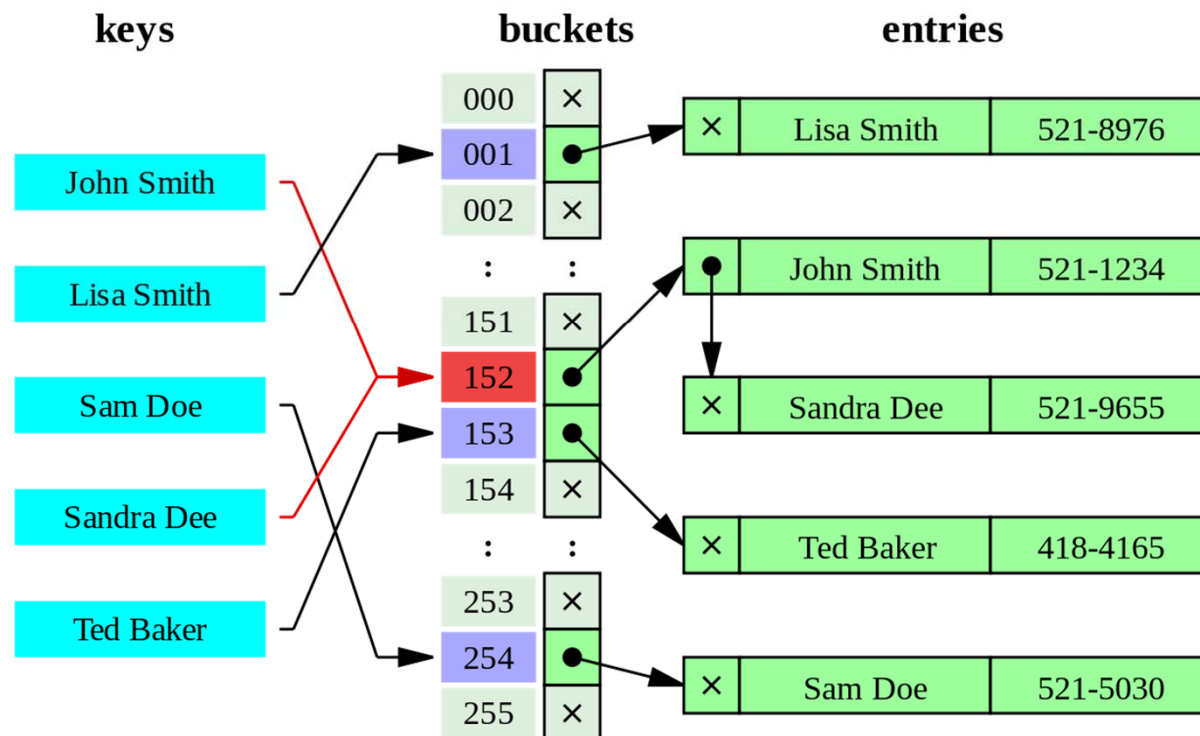
Колизии са на практика неизбежни, когато хешираме произволно подмножество от голям набор от възможни ключове. Ето защо повечето имплементации на хеш таблици имат някаква стратегия за справяне с такива събития. Всички тези методи изискват ключовете (или препратки към тях), да се съхраняват в таблицата заедно със свързаните с тях стойности.



Тема 10 – Алгоритми за хеширане

Методи за справяне с колизии:

Нареждане в списък (chaining) - Най-разпространеният начин за решаване на проблема с колизиите е нареждането в списък (chaining). Той се състои в това двойките ключ и стойност, които имат еднакъв хеш код за ключа, да се нареждат в списък един след друг.



Тема 10 – Алгоритми за хеширане

Методи за справяне с колизии:

Отворена адресация (open addressing) - Тази стратегия е алтернативна на нареждането в списък. Тук идеята е, че в случай на колизия се опитва да се сложи новата двойка на някоя свободна позиция от таблицата. Методите се различават по това как се избира къде да се търси свободно място за новата двойка. Освен това трябва да е възможно намирането на тази двойка на новото ѝ място. Този тип методи са неефективни при голяма степен на запълненост.

